NirmanIQ Internship — Day 15 Checkpoint Assessment #1

DELIVERABLE
Mini Full-Stack App ("Tower Progress Tracker")

TECH STACK
Next.js 14 App Router + NestJS + PostgreSQL + TypeORM

AUTHOR
Penumuru Madhu Sudhan Reddy

Executive Summary: Checkpoint Assessment #1 validates end-to-end integration across Phase 1 (Days 1–15). It connects a Next.js App Router client to an enterprise NestJS backend backed by PostgreSQL. The app implements dual-role workflows: Project Managers (PM) oversee projects and configure towers, while Site Engineers (SE) submit ground-level floor completion metrics through authenticated progress update modals.

1. Role-Based Permissions & Enterprise Boundaries

Route	Method	Site Engineer (SE)	Project Manager (PM)	Enforcement Mechanism
`/auth/login`	POST	Allowed	Allowed	Public endpoint; bcrypt password verification
`/projects`	GET	Allowed	Allowed	`JwtAuthGuard` (Stateless Bearer JWT token)
`/projects`	POST	**403 Forbidden**	Allowed	`RolesGuard` with `@Roles(Role.PM)`
`/projects/:id/towers`	POST	**403 Forbidden**	Allowed	`RolesGuard` with `@Roles(Role.PM)`
`/floors/:id/progress`	PATCH	**Allowed**	Allowed	`RolesGuard` with `@Roles(Role.SE, Role.PM)`

2. Construction Domain & Risk Recalibration Model

Each tower's risk status is computed dynamically based on the average completion percentage of its child floors:

Average Completion	Risk Level	Badge Display	Color Token	NirmanIQ Domain Definition
≥ 60%	Level 0	ON TRACK	`#10b981`	Slab and structural work aligned with milestone target.
25% – 59%	Level 1	AT RISK	`#f59e0b`	Minor schedule slippage or material supply lag.
< 25%	Level 2	HIGH RISK	`#ef4444`	Critical milestone failure requiring PM intervention.

3. Full-Stack Data Flow & State Synchronization

Frontend Caching (React Query)

```
// TanStack React Query Cache Invalidation
const updateMutation = useMutation({
  mutationFn: ({ floorId, percentage }) =>
    apiUpdateFloorProgress(floorId, percentage),
  onSuccess: () => {
    // Invalidate tower and project queries
    queryClient.invalidateQueries({ queryKey: ['tower'] });
    queryClient.invalidateQueries({ queryKey: ['project'] });
    toast.success('Floor progress updated');
  }
});
```

Backend Progress Recalibration

```
// Recalibrate tower risk in TypeORM
const avg = floors.reduce(
  (acc, f) => acc + Number(f.completionPercentage), 0
) / floors.length;

let newRisk = 0;
if (avg < 25) newRisk = 2;
else if (avg < 60) newRisk = 1;

await this.towerRepo.update(floor.towerId, {
  riskLevel: newRisk,
  updatedAt: new Date()
});
```

4. Execution & Testing Instructions

Run the automated test runner to verify full-stack compliance across all 5 grading criteria:

```
# Run automated integration test suite:
npm run day15:test

# Start NestJS Backend API (Port 3015):
npm run day15:backend

# Start Next.js App Router Frontend (Port 3050):
npm run day15:frontend
```